

- Multiple calls to pure functions can run on parallel CPU cores or in a multithreaded program because they don't have to depend on any external resources that require them to be run in any particular sequence.

You can and should write pure functions in Python whenever possible. Python functions are made pure by convention only; there's no setting that causes the Python interpreter to enforce purity. The most common way to make your functions pure is to avoid using global variables in them and ensure they don't interact with files, the internet, the system clock, random numbers, or other external resources.

Higher-Order Functions

Higher-order functions can accept other functions as arguments or return functions as return values. For example, let's define a function named `callItTwice()` that will call a given function twice:

```
>>> def callItTwice(func, *args, **kwargs):
...     func(*args, **kwargs)
...     func(*args, **kwargs)
...
>>> callItTwice(print, 'Hello, world!')
Hello, world!
Hello, world!
```

The `callItTwice()` function works with any function it's passed. In Python, functions are *first-class objects*, meaning they're like any other object: you can store functions in variables, pass them as arguments, or use them as return values.

Lambda Functions

Lambda functions, also known as *anonymous functions* or *nameless functions*, are simplified functions that have no names and whose code consists solely of one `return` statement. We often use lambda functions when passing functions as arguments to other functions.

For example, we could create a normal function that accepts a list containing a 4 by 10 rectangle's width and height, like this: